The Harness Moves to the Center

Five agentic-engineering papers on the scaffolding around the model — a reader's guide

🤖 Today's Agentic Engineering Papers · digest of 2 June 2026

June 2, 2026

Chapter 0: The Harness Moves to the Center

For a decade the reflex in this field was simple: if the agent isn't good enough, get a better model. The five papers in this digest, read together, describe a quieter shift in where the leverage now sits. Four of them have the word *harness* in the title, and that's not a coincidence of the week's arXiv listings — it's the through-line. The harness, the engineered environment wrapped around a frozen model (its prompts, tools, memory, state, and the loop that drives them), has become a first-class surface that engineers design, train, and measure on its own terms. In several of these results it carries more of the performance than the model does.

The cleanest statements of that claim are almost startling as numbers. In **Crafter**, swapping the underlying image generator from one frontier model to a stronger one moves the final score by a fraction of a point, while the multi-agent scaffolding around it adds sixteen to twenty-two. In **Harness-1**, a 20-billion-parameter search agent holds its own against frontier retrievers many times its size, and its biggest gains land on the *held-out* benchmarks it never trained on — the tell that it learned operations over a general search interface rather than memorizing domains. **Harness**

Updating drives the point home from the budget side: the model that writes good lessons and the model that lives by them are different jobs, and only one needs to be expensive. The conviction these papers independently arrive at is that scale is no longer the only dial, and often not the highest-leverage one.

Get the state out of the transcript

If there's one mechanical idea to carry into all five chapters, it's that an append-only conversation is a bad place to keep an agent's working state. Each paper, in its own domain, pulls structure out of the raw context and into something the agent operates on deliberately. Harness-1 replaces the growing transcript with a rendered working memory — a candidate pool, a curated set with eviction, an evidence graph — that the policy *edits* rather than re-derives every turn. Crafter threads a shared, evolving specification between its agents and insists that revisions arrive as **typed edits**, not free-text addenda that quietly accumulate contradictions. SkillAdaptor maintains an external skill library that it grows from experience. In every case the win comes from giving the model a clean, structured surface instead of asking it to reconstruct order from a flat scroll of history. When you read these chapters, watch how much of each system's intelligence lives in the *shape of the state* rather than in the prompt.

Trust nothing you haven't gated

The second shared thread is a discipline: these systems don't accept their own updates on faith. SkillAdaptor's whole contribution is precision plus a gate — localize a failure to a single step, edit only the skill responsible, then **replay** the candidate library against the current one and reject anything that doesn't measurably help. Its ablations show the gate isn't decoration; remove it and the system gets jittery, variance balloons, quality decays over long horizons. Crafter independently lands on the same instinct — a best-so-far checkpoint with revert-on-regression, because iterative LLM editing is empirically non-monotonic — and pairs its language-model critic with *programmatic checkers* for the things code verifies better than judgment: text overflow, arrow endpoints, missing parts. Harness-1 bakes a verify action into the loop. The recurring lesson: self-improving and self-correcting agents drift into self-harm without an acceptance check, and wherever a deterministic checker can validate a slice of the output, it should.

Capability is uneven, so spend unevenly

A subtler thread runs between **Harness Updating** and **Harness-1**: model capability doesn't translate uniformly into agent performance, which means budget shouldn't be spread uniformly either. Harness Updating finds that writing useful harness updates is cheap and flat across model tiers — a 9B model matches Opus — while *benefiting* from updates is non-monotonic, peaking in the mid-tier. The practical reading is to put your money on the agent that solves the task, run a cheap evolver, and

treat “can this model reliably invoke and adhere to its harness” as a skill to verify rather than assume. Harness-1 is the existence proof from the other direction: a small model, given a well-designed interface and the right training tricks, beats far larger ones. Bigger is not the lever; *fit between model and harness* is.

And keep the ruler honest

None of this matters if you can’t measure it, which is why **A Matter of TASTE** belongs in the bundle even though it builds no agent. Its uncomfortable finding — that a high score on a maturing benchmark increasingly signals saturation, not competence — is the meta-condition for everything else here. SkillAdaptor’s honest, modest one-point gains are only legible on a benchmark that hasn’t run out of room. TASTE offers a cheap, repeatable way to keep manufacturing harder, broader tasks as fast as agents climb, by building tasks outward from diverse tool sequences instead of inward from a human’s pet scenarios. Read it as the floor the other four stand on: the ratchet that keeps their improvements visible.

Carry these four threads into the chapters — state pulled out of the transcript, updates that must pass a gate, capability spent where it actually pays, and a benchmark sharp enough to register the difference. The model still matters. But this week’s papers agree, from five different directions, that the engineering has moved into the harness around it.

Chapter 1: Harness-1

Paper: Harness-1: Reinforcement Learning for Search Agents with State-Externalizing Harnesses · arXiv 2606.02373 · **Code:** github.com/pat-jj/harness-1

Most search agents are trained as a policy over a growing transcript. The model issues a query, reads what comes back, decides what's missing, and searches again — and the whole time it's also supposed to remember which documents it has seen, which candidates are worth keeping, which constraints are still open, and which claims it has actually checked. That's two jobs crammed into one network: deciding *what to search* and keeping the books on *everything it has searched so far*. Harness-1's core argument is that the second job doesn't belong in the model at all.

Move the bookkeeping out of the model

The team built a 20-billion-parameter retrieval agent (on top of gpt-oss-20b) that runs inside what they call a **stateful harness** — an environment that maintains the agent's working memory for it. Instead of re-deriving the state of the search from a raw, append-only transcript every turn, the policy sees a clean, rendered **working memory**: a candidate pool, an importance-tagged curated set, a compact evidence graph, verification records, search history, and a context-budget marker. The model's actions don't just extend a transcript — they *edit this state*. A `curate` action adds, removes, and importance-tags documents. A `verify` action checks a claim the model writes against documents it already has. A `review docs` action re-renders evidence it has seen without spending another corpus call.

The division of labor is the whole idea. The policy keeps the semantic decisions: what to search, which documents to promote or discard, what to verify, when to stop. The harness handles the recoverable, mechanical stuff: deduplicating results, compressing search hits down to the four most relevant sentences, capping the curated set at 30 and evicting the lowest-importance documents when it overflows, and building an evidence graph by regex-scanning each chunk for proper nouns, years, and dates so the model can ask "what have I seen about Brussels?" as a lookup instead of a full-transcript reread. They call this principle **stateful cognitive offloading**: give reinforcement learning a stable interface to improve search behavior over, rather than forcing it to rediscover bookkeeping from scratch on every rollout.

Why a rich interface needs careful training

A fancier harness doesn't automatically make a trainable environment, and the paper is candid about the traps. If every rollout starts from an empty curated set, hard queries hand back near-identical zero rewards and RL learns nothing. If the derived state is too verbose, it crowds out the actual evidence. And if the reward only prizes discovery, the policy learns to spam searches and skip

curation entirely. Harness-1's fixes are worth stealing on their own: **warm-started curation** auto-seeds the curated set with the top eight reranked results from the first successful search, so early trajectories differ in how they *refine* rather than whether they found anything; compact derived-state rendering keeps the interface readable; and a tool-diversity reward keeps the agent from collapsing into search-only mode.

That last one is the most instructive. The authors ran two identical RL setups differing only in whether the diversity bonus was active. Without it, the agent quickly learned to fire off parallel searches, drove trajectory recall up, and bypassed curation and verification — tool diversity fell from about six tools to three and a half, and curated recall plateaued around 0.53. With the bonus, the agent kept combining search with curation and verification, and final recall climbed to roughly 0.60. The harness exposing useful tools wasn't enough; the reward had to make those tools worth using.

The numbers, and the part that actually matters

Across eight benchmarks spanning web, finance, patents, and multi-hop QA, Harness-1 hits **0.730 average curated recall** — beating the next-best open search subagent by 11.4 points and edging out much larger frontier retrievers including GPT-5.4, Sonnet-4.6, Kimi-K2.5, and a 120B model, with only Opus-4.6 ahead on average. A 20B model holding its own against frontier searchers is the headline, but the diagnostic result is transfer. The SFT data covered four benchmark families; the four *held-out* benchmarks never touched training. Standard intuition says gains should concentrate near the training data. The opposite happened: Harness-1 gained +7.9 points on average on the source families and **+17.0 points on the held-out ones** — a 2.2x larger lift on the tasks furthest from anything it trained on.

That's the tell that the harness is doing real work. The policy isn't memorizing domain patterns in its weights; it's learning *operations over a domain-general search state* — refine the seeded set, read bridge entities off the evidence graph, re-inspect uncertain candidates, verify before promoting, submit a compact set. Those operations travel. An ablation study reinforces it: strip out the harness mechanisms one at a time and recall degrades as the trained policy reverts to a wide, shallow, search-dominated mode; disable them all and recall drops 12.2%, the largest hit of any single change. The harness, in other words, is where per-turn search bandwidth gets converted into a discriminative final set.

What an engineer could borrow

The practical lesson cuts against the reflex to make agents smarter by making them bigger or by piling more reasoning into the context window. If you're building a search or research agent, the highest-leverage move may be to pull state management *out* of the model and into the environment. Concretely, you could externalize working memory into structured tools rather than letting it accumulate in the scratchpad: a candidate pool you dedup and compress before it ever reaches the prompt, a curated set with explicit importance tags and an automatic eviction policy, a verification

cache that checks claims against documents you already hold, and a lightweight entity graph that turns “what have I seen about X” into a lookup. You don’t need RL to benefit from most of this — even a prompted agent gains a cleaner decision surface and a smaller, less distractible context. And if you do train, the warm-start and diversity-reward tricks are the difference between a harness that *could* help and one the policy actually learns to exploit. Interface design, not just model scale, is a first-class lever for search-agent performance.

That framing — that the harness around the model can drive performance as much as the model itself — sets up an uncomfortable question the next chapter takes head-on. If a good harness is so powerful, which models can actually *produce* good harness updates, and which can actually *benefit* from them? The next paper, “Harness Updating Is Not Harness Benefit,” pries those two abilities apart in a measurement study of self-evolving agents, and finds they don’t track together at all: the ability to generate useful harness updates stays roughly flat across model tiers, while the ability to benefit from them peaks in the mid-tier — a result that complicates the tidy story of bigger models getting more from better scaffolding.

Chapter 2: Harness Updating Is Not Harness Benefit

Paper: Harness Updating Is Not Harness Benefit: Disentangling Evolution Capabilities in Self-Evolving LLM Agents · arXiv 2605.30621 · **Code:** github.com/A-EVO-Lab/a-evolve

A 9-billion-parameter open model and Claude Opus 4.6 are asked the same thing: watch an agent fumble a task, then write a reusable skill that fixes the fumble. You would expect the frontier model to win. It doesn't. On one benchmark the tiny model writes the *better* skill, and when researchers lined up the two skills side by side, they were nearly the same recipe — same sequence of steps, differing only in verbosity. That single result quietly rearranges how you should think about spending money on a self-evolving agent.

Two jobs that look like one

Most self-evolving agents share a setup worth naming up front. The model weights are frozen. What changes is the **harness** around the model: the prompts, skills, memories, and tool descriptions that shape how it reads a task, acts, and recovers from mistakes. A self-evolving agent runs tasks, watches what went wrong, and writes lessons back into that harness so the next run goes better. The thing that does the writing is called the **evolver**.

The usual way to evaluate one of these systems asks a single end-to-end question: did the agent get better? Useful, but it blurs two very different capabilities into one number. One is **harness-updating** — being good at turning execution evidence into a genuinely useful, persistent update. The other is **harness-benefit** — being good at actually exploiting an updated harness while solving the task. The paper's whole move is to pull these apart, pairing seven models as both evolvers and solvers across three agentic benchmarks (software engineering, real MCP tool use, and skill-based execution) to see which capability tracks raw model strength.

The answer to both halves is counterintuitive, and that's where the value is.

Writing good updates is cheap

Fix the agent doing the work, swap only the evolver, and the gains barely move. Across every benchmark, the gap between the best and worst evolver was at most 3.1 percentage points — and no single model won everywhere. Qwen3-235B topped the software-engineering benchmark and came in dead last on tool use. The smallest model in the study, Qwen3.5-9B, posted the *highest* gain on the skills benchmark. Model scale simply didn't predict who wrote better updates.

Why would distilling a lesson be so much easier than acting on one? Because the evolver has the easy job. It gets to look back at a completed trajectory — including the failures — and summarize what should have happened. That's a bounded, hindsight-driven writing task, and even a small model can spot "you forgot to filter for the FINISH event" and write it down clearly. The skill the 9B model produced wasn't a watered-down version of Opus's; it was procedurally the same artifact.

The practical reading: scaling up your evolver buys you almost nothing. If you're paying frontier prices to generate harness updates, that budget is being wasted on the part of the loop that was never the bottleneck.

Benefiting from updates is where models diverge — strangely

The other capability behaves nothing like the first. Harness-benefit is **non-monotonic** in base capability. Weak models gain little. Mid-tier models gain the most. Strong models gain less than mid-tier. On the software benchmark the peak sat at Qwen3-235B with a 19.3-point jump, while stronger Opus gained only 2.6 and weaker Qwen3-32B only 4.4. On tool use the peak moved to GPT-OSS-120B at 7.0 points.

The strong end is easy to explain — a ceiling effect. Frontier models already solve most tasks bare, so there's little headroom for a skill to add. The weak end is the surprise. Those models have the *most* room to improve, yet they capture the least of it. The paper traces this to two distinct failure modes, and the distinction matters because they call for different fixes.

The first is **harness activation failure**: the model never gets the skill into context at all. Qwen3-32B loaded a relevant skill in only 25% of its trajectories, versus roughly 96% for strong models. In one case it correctly identified the right skill but smuggled the load request inside a larger action blob instead of issuing it as a clean call — and the runtime's format gate rejected it. The lesson was sitting right there and never entered the working context.

The second is **harness adherence failure**: the skill loads, and the model still doesn't follow it. Qwen3-235B is the clean illustration — it loaded skills 96% of the time, nearly matching Opus, yet followed the loaded guidance only 35% of the time against Opus's 76%. In one walkthrough, a model loaded a skill that spelled out a five-engine text-to-speech fallback chain, treated it as a one-shot script, hit the first failure, and gave up — never trying the alternatives the skill explicitly listed. Adherence also *decays over the trajectory*: weak models started around 0.52 adherence right after loading and drifted to 0.13 by the final step, while Opus held from 0.89 to 0.80. The weakness isn't misreading the harness at load time; it's losing the thread over a long task.

What an engineer building these agents should do with this

Three concrete moves fall out of the split.

First, **put your capability budget on the solver, not the evolver.** Post-evolution performance varies far more with which model solves the task than with which model wrote the update — within-agent spread across seven evolvers topped out at about 5 points, dwarfed by a 36-point gap between strong and weak solvers' base capability. If you're cost-constrained, run a cheap, even tiny, evolver and pour the savings into a stronger solving agent. The reverse — premium evolver, budget solver — is the trap.

Second, **treat harness invocation as a skill to train, not a behavior to assume.** A model that can't reliably emit a clean skill-load call gets nothing from your beautifully evolved harness. Before chasing better update quality, audit your load rate: are relevant artifacts actually reaching context? A malformed call that a format gate silently rejects looks identical to "the harness didn't help" in an end-to-end score, and you'll debug the wrong half of the loop.

Third, **measure adherence as a curve, not a checkbox.** Loading isn't following, and following at turn one isn't following at turn twenty. The paper's phase-level scoring — adherence right after load, at the midpoint, at final validation — is a cheap diagnostic you can borrow directly. If your agent drifts off its own loaded guidance late in long-horizon tasks, the fix is sustained instruction-following, not more skills.

The broader takeaway is a reframing of the whole self-evolution stack: the model that writes the lessons and the model that lives by them are doing genuinely different jobs, and only one of them needs to be smart. Optimize them separately, or you'll spend frontier money where a 9B model would have done.

That insight is about a *single* agent learning to use a harness over time. The next chapter turns to coordination across *many* specialized agents at once — Crafter, a multi-agent harness that takes heterogeneous inputs like text, raw data, and rough sketches and routes them through role-specialized agents to produce editable, publication-quality scientific figures. Where this chapter asked which model should hold the budget, the next asks how to divide the work.

Chapter 3: Crafter

Paper: Crafter: A Multi-Agent Harness for Editable Scientific Figure Generation from Diverse Inputs · arXiv 2605.30611 · **Code:** github.com/HaozheZhao/Crafter

Anyone who has built on top of a raw image model knows the failure mode: you ask for a clean architecture diagram, get back something with a garbled label and a misaligned arrow, retype the prompt, and get back a *different* set of mistakes. Naive retry doesn't converge. Pile up free-text corrections — "enlarge the title," then "reduce the white space" — and the model silently absorbs the contradictions until faithfulness quietly rots. Crafter's central claim is that this isn't a model problem. It's an orchestration problem, and the fix is a harness, not a bigger backbone.

The harness, not the model

The headline result for an engineer: wrapping an off-the-shelf image generator in a structured multi-agent loop beat the strongest agentic baseline by 16.6 points on one benchmark and 22.2 on a tougher, broader one — and it did so while leaving the *generator itself* untouched. Swap the backbone from Nano Banana 2 to Nano Banana Pro and the overall score barely moves (0.3 to 2.1 points). The lift comes from the scaffolding around the model. That's the whole thesis stated as a number: the harness raises the floor and the consistency of the pipeline, not the ceiling of the generator.

Crafter formalizes the harness as a four-role loop over a shared, evolving **specification** — a structured record that holds the current plan, the revision history, and prior diagnostics. Each round, a **designer** proposes a plan, an **executor** renders it, a **verifier** inspects the artifact and emits a diagnostic, and a **reviser** writes edits back into the spec. Five concrete agents fill those four roles: an intent reasoner that reads the input (a paper, a sketch, a reference image) and seeds the initial spec, a plan generator, the image-generation backend, a critic, and a specification refiner — with a convergence judge governing whether each round is accepted, refined further, or reverted.

Three design choices make the loop actually work, and each is independently load-bearing — removing any one costs 5 to 9 points in ablation.

Three moves worth stealing

Branch at the plan level, not just the refinement level. Image generators have high variance on structured layouts: the same prompt yields qualitatively different compositions across seeds. So the designer proposes K candidate framings in parallel (banner, multi-column grid, numbered-step sequence — from an 11-key style vocabulary), renders them all, and the judge picks the best as the starting point. The insight is that a bad compositional choice early — rendering a comparison as a

block diagram — poisons every refinement round downstream. Branching escapes that *before* you spend rendering budget refining a doomed layout. Killing this move was the single most expensive ablation.

Accumulate typed edits, not free text. This is the move most directly transferable to any iterative-generation agent. Instead of appending natural-language corrections to a growing prompt, the reviser converts each diagnostic into structured operations from a fixed vocabulary — add a layout constraint, ban an artifact category, resize a named element — applied *in place* on the shared spec. The next prompt is rebuilt from that coherent record, not from a teetering stack of amendments. Replacing typed edits with free-text revision cost 8.9 points, the largest single drop, and it validates the original diagnosis: free-text contradictions degrade output without any one round looking wrong.

Make the critic directive, not scalar. A "5/10" score gives the reviser nothing to act on. Crafter's critic instead emits per-dimension scores across six axes plus an explicit defect list and suggested corrections. The reviser reads the issues and suggestions, not the numbers. Pair that with a verify-then-refine loop (capped at three rounds) that keeps a best-so-far checkpoint and reverts whenever a round regresses — because LLM-driven editing is empirically non-monotonic, a detail worth internalizing for anyone running self-correction loops.

Why the output stays editable

The other half of the contribution is that the result doesn't have to stay a flat raster. **CraftEditor** reuses the exact same four-role harness to convert a raster figure into a coordinate-faithful, editable **SVG** — the kind of artifact you can open in standard vector tooling and actually revise: swap an icon, recolor a scheme, complete a partial diagram. It runs three phases. Extraction replaces brittle segmentation with an instruction-driven cleaning loop (a vision agent authors a keep/delete plan, an image editor executes it, a verifier checks the canvas). Processing captions and classifies each isolated element. Composition assembles an SVG skeleton and refines it through a **hybrid critic** — a vision-language model judging global layout fidelity alongside *programmatic checkers* that audit the things VLMs miss: text overflow, arrow-endpoint accuracy, element overlap, missing components.

That hybrid critic is the quietly clever part. The largest editing gains landed on exactly the structural axes — text and arrows — where deterministic code catches what a language model's eyeball glosses over. CraftEditor scored 8.04 overall against 6.91 for the next-best raster-to-vector system. The lesson generalizes past figures: when you can verify part of a structured artifact with a real checker, do it, and reserve the model's judgment for what only judgment can assess.

What an engineer could try

The reason this paper matters beyond scientific figures is right there in the conclusion: because the harness is executor-agnostic, the authors expect the pattern to carry into any structured-output domain. If you're orchestrating role-specialized agents on a task where the output is a *composition of*

discrete parts with constraints between them — SQL, infrastructure-as-code, UI layouts, slide decks, structured documents — the borrowable kit is concrete. Keep an evolving structured spec as the shared memory instead of threading raw conversation between agents. Let agents write typed deltas to that spec, never free-text addenda into each other's prompts. Branch at the planning level to dodge unrecoverable early choices. Make your critic hand back specific, per-dimension defects, not a score. Checkpoint and revert, because iterative LLM editing backslides. And wherever a deterministic checker can verify a slice of the output, wire it in alongside the model judge. Notably, all the task-specific behavior here lives in agent *prompts*, not architecture — adding a whole new input condition took one branch in the instruction builder and one table entry, no pipeline rewrite.

That portability — a fixed harness, swappable executor, behavior carried in prompts — is also what sets up the next question. Crafter's agents share memory but don't *learn* from their own runs; each task starts fresh. The next chapter turns to SkillAdaptor, a training-free framework that lets an agent mine its own past trajectories to synthesize reusable skills and keep adapting them — and crucially, attributes failure at the step level rather than blaming a whole trajectory, so the lessons it carries forward are precise rather than blunt.

Chapter 4: SkillAdaptor

Paper: SkillAdaptor: Self-Adapting Skills for LLM Agents from Trajectories · arXiv 2606.01311 ·
Code: github.com/zjunlp/SkillAdaptor

A long-horizon agent run fails on step nineteen. Somewhere back at step six it parsed a spreadsheet wrong, and everything after that was wasted motion built on a bad number. The conventional way to learn from that failure is to hand the whole transcript to a model and ask “what should we do differently next time?” The model, staring at nineteen steps where eighteen looked fine, tends to rewrite half the playbook. SkillAdaptor’s argument is that this is exactly the wrong instinct. The failure was one step. The fix should be one skill. Everything else should be left alone.

Fix the step, not the trajectory

The core move here is precision in blame. Most training-free skill-improvement pipelines — the ones that let you upgrade a frozen model by editing an external library of reusable “skills” rather than retraining weights — learn from trajectory-level or session-level outcomes. They treat the completed run as the unit of reflection. SkillAdaptor treats the *step* as the unit, and it does so with an explicit credit-assignment pipeline that runs in three named stages.

First, **attribution**. A Localizer reads the failed trajectory and predicts the earliest accountable fault step — the first place where, had the agent done something different, the outcome would have changed. In the paper’s worked example, a CSV-and-Excel summarization task fails at the final step, but the Localizer pins the real fault back at interaction step six: the agent finalized after passing only partial validation checks. A Linker then takes that fault step and assigns responsibility *weights* to the skills that were actually in play, producing a ranked suspect list rather than a vague gesture at the library. It also decides whether this is a case of a skill that misled the agent (route to REVISE) or a capability the library never covered (route to GENERATE).

Second, **modification**. If the verdict is REVISE, the highest-weighted skill gets a minimal, additive edit — a new precondition, a disambiguation, an anti-loop guard — and replaces the old card. If it’s GENERATE, a fresh skill is synthesized from the localized context and appended, then deduplicated against the existing library so you don’t accumulate near-identical cards.

Third — and this is the part most worth stealing — **qualification**. No edit is trusted on faith. Before a candidate update is committed, the framework re-runs tasks under both the old library and the proposed new one and compares outcomes. The update is accepted only if performance doesn’t drop. If it does, the modification is thrown away and the original library is kept untouched. This acceptance gate is what keeps continual adaptation from drifting into self-inflicted decay over long horizons.

What the gate actually buys you

The ablations make the case better than the headline numbers do. Strip out the Localizer and Linker and WebShop success falls from 33% to 28.6% — step-level targeting is doing real work, not decoration. But the more telling result is what happens when you remove the qualifier gate. Accuracy sags, but the real damage shows up in *variance*: PinchBench spread balloons from ±5.2 to ±8.1, WebShop success spread nearly doubles. Without the acceptance check, bad skills slip into retrieval and the whole system gets jittery. And when you keep only the skills distilled from successful runs — no failure-driven attribution at all — the gains essentially vanish back to the bare-model baseline. The lesson lands hard: skills mined only from your wins don't transfer. The instructive signal lives in the failures, but only if you can localize it.

Across WebShop, PinchBench, and Claw-Eval, run on Kimi-K2.5, GLM-5, and GPT-5.2, SkillAdaptor beats both no-skill baselines and prior skill-adaptation methods on every suite. The margins are modest — roughly a point to a point-and-a-half on the headline metrics — but they're consistent across three different backbones, which is the harder thing to achieve. There's a quieter efficiency story too: skills front-load context, so per-task input tokens rise, but the agent takes *fewer* interaction steps (Claw-Eval drops from 7.3 to 5.8), trading repeated environmental flailing for richer in-context reasoning. The authors are careful that this isn't a free lunch on cost — it's a redistribution of where the compute goes.

What an engineer could borrow

You don't need to adopt this whole framework to get value from it. The reusable idea is that you can improve a long-horizon agent *without retraining anything* by maintaining a skill library and editing it from failures — and the editing discipline is the part that matters. Most teams running agent harnesses already log failed trajectories and let them rot. The borrowable pattern: when a run fails, don't ask a model to "summarize lessons learned" over the whole transcript. Ask it for the *first* actionable fault step, link that step to the specific instruction or skill card that steered the agent wrong, and edit only that card. Then — the non-negotiable part — gate the edit behind a replay check that compares the candidate library against the current one and rejects anything that doesn't measurably help. That acceptance gate is cheap to bolt onto an existing eval harness and it's the single thing standing between "self-improving agent" and "library that slowly poisons itself."

It works best where failures expose observable intermediate structure — code, data, DevOps tasks where a wrong parse or a bad argument is visible in the trace. It's weaker where failure depends on persistent state, external knowledge, or cross-session memory that no single step-level edit can repair. Knowing that boundary tells you where to point this kind of self-adaptation and where to reach for something else.

Of course, none of this self-improvement means anything if your benchmark can't tell good agents from bad ones in the first place. Once leaderboards saturate and easy tasks dominate the score, a

method that wins by a point becomes indistinguishable from noise — and that's precisely the problem the next chapter takes on. A Matter of TASTE turns the lens around to the benchmarks themselves, attacking coverage gaps and easy-task bias so the evaluations stay sharp enough to register exactly the kind of improvement SkillAdaptor is chasing.

Chapter 5: A Matter of TASTE

Paper: A Matter of TASTE: Improving Coverage and Difficulty of Agent Benchmarks · arXiv 2605.28556

A leading model scores 0.94 on a respected agent benchmark. You read that as “this model has nearly solved customer-support agent tasks.” But put the same model on a harder, broader version of the same domain and it falls to 0.61. The capability didn’t change. The benchmark just ran out of room to measure it.

That gap is the whole story of this paper, and it has a blunt lesson for anyone who builds or trusts agent leaderboards: a high pass rate on a maturing benchmark increasingly means the benchmark is saturated, not that the agent is good. The authors built a way to manufacture harder, more varied tasks automatically, and when they did, the scores of “nearly solved” models cratered.

The bias hiding in how we write agent tasks

Most tool-using-agent benchmarks are authored the same way. A human writes a natural-language scenario (“a customer wants to rebook a canceled flight”), then works out the sequence of tool calls that would satisfy it. It’s slow, expensive, and error-prone — earlier work found dozens of broken tasks in the popular τ-Bench that had to be patched in a “Verified” rerun.

But the deeper problem is what this authoring style *samples*. Because the tool sequence is only ever derived as an afterthought, benchmarks end up exercising whatever tool combinations happened to occur to the author. The space of tool-use patterns an agent could be tested on is vast; hand-authored tasks cover a thin, lopsided slice of it. A benchmark can look big while testing the same few procedural shapes over and over.

The authors name three things a good agent benchmark needs. **Validity**: every task must be automatically checkable and actually solvable. **Difficulty**: tasks should separate strong agents from weak ones rather than letting everyone pass. And **coverage**: tasks should span structurally different patterns of tool use, not pile up near-duplicates. Coverage is the one nobody had pinned down — so they defined it concretely as the diversity of *tool sequences*, the ordered list of which tools a correct solution calls. That definition is the hinge for everything that follows.

Reverse the process: start from the tools

The core move in **TASTE** — Task Synthesis from Tool Sequence Evolution — is to flip task construction around. Instead of writing a story and recovering its tool calls, you first sample a diverse spread of valid tool sequences, then build full tasks around each one. If you can explore the tool-

sequence space directly, you can deliberately cover it instead of hoping an author wandered into the interesting corners.

It runs in three stages. The first trains what they call an **Adaptive Contrastive n-gram model** over tool sequences — a deliberately simple machine that predicts the next tool from the previous couple of tools. Two twists make it work. It's *adaptive*: it samples candidate sequences, has an LLM judge whether each could plausibly solve a real scenario, and folds that verdict back in to sharpen its own distribution. And it's *contrastive*: it keeps separate tallies for patterns seen in plausible versus implausible sequences, so it actively learns to avoid nonsense like modifying a reservation after canceling it. That negative signal matters — naive uniform sampling produced valid sequences only 6.7 percent of the time, while the full trained model hit 86.7 percent. Roughly a thirteen-fold lift, from a model cheap enough to run for about ten dollars a domain.

The second stage prunes the thousands of candidates down to a representative set. It clusters them with a distance metric that understands tools — swapping two similar search functions costs little, swapping a read for a write costs a lot — then picks the center of each cluster. You get sequences that are genuinely structurally distinct rather than a pile of trivial variations.

The third stage turns each chosen sequence into a real task: an LLM invents a plausible scenario and the database records to support it, a rule-based pass checks the task is structurally sound, and a "verifier agent" tries to actually solve it (given shuffled, partly-masked hints so it can't just replay the answer). Only tasks the verifier can complete survive — and that gate is tuned for near-perfect precision, so anything passing is almost certainly solvable. Then comes the part that makes tasks *hard*: an evolution step that makes the simulated user vaguer and less cooperative, and salts the database with decoy records — a flight on the right route with no seats left — designed to trip up agents that aren't reading carefully.

What the harder benchmark exposed

Running TASTE on three τ²-Bench domains produced τc-Bench, then they evaluated eleven agent/user model pairings on it. Scores dropped across the board, by anywhere from 5 to 80 percent. The fall was steepest exactly where you'd worry most: the models that looked nearly finished on the old benchmark. Gemini-3-Flash went from the 0.82–0.94 range down to 0.28–0.61. Crucially, the new tasks were also genuinely *broader*, not just meaner — the diversity of tool combinations agents had to execute more than doubled by the authors' coverage measures. And when they hand-checked the handful of tasks every single agent failed, the tasks themselves were valid. The agents simply couldn't do them.

Reading the result

The takeaway for practitioners is uncomfortable in a useful way. When a benchmark's top scores cluster up near the ceiling, your instinct should not be "the models are great" but "this benchmark has

stopped telling me anything.” Saturation and competence look identical from the leaderboard. The only way to tell them apart is a harder test.

If you build or rely on agent evals, there are concrete things to borrow here. Audit your benchmark for coverage, not just headcount: count how many genuinely distinct tool-call patterns your tasks exercise, because a hundred near-duplicate tasks measure one thing a hundred times. Treat hard, distractor-laden tasks as the signal — a decoy record or an uncooperative user is often where real agents break, and where leaderboard-toppers separate from the pack. And consider building your tasks from the tool sequences outward rather than from a story inward, so coverage is something you design for instead of something you hope you stumbled into. You don’t need this exact pipeline to act on the idea; even manually, you can ask whether your tasks span the procedural space or just the scenarios you happened to imagine.

The larger point is that benchmarks have a shelf life, and the better models get, the shorter it grows. What TASTE really offers is not one harder benchmark but a cheap, repeatable way to keep making the next one — to ratchet difficulty and coverage upward as fast as the agents climb. The difficulty knobs are still in your hand: longer sequences and more write-heavy tasks both reliably drove scores down, which means there’s headroom to make tasks harder still when today’s batch gets solved. In a field where a model can go from impressive to obsolete in a quarter, an evaluation you can regenerate on demand may matter more than any single score it produces.
